Faster-than-Light: Latency Measurement Artifacts in Streaming Benchmarks

Gustavo Pedro Ricou, Romaric Duvignau, Nikolas Herbst, and David Gregg

***Abstract*—Message-broker benchmarks now compare systems on sub-millisecond paths, where the delivery measured is shorter than the delays and resolution of the timestamps that measure it. A widely used benchmark misreports in two ways that leave no trace. First, a latency computed from two timestamps written by different threads is negative whenever the thread writing the reference timestamp is scheduled late by more than the delivery. A per-run sign check rejected $1{,}321$ of our $2{,}266$ runs; referenced to the send timestamp the same events never invert on one clock by construction, so the cause is scheduling, not clock synchronization; real-time priority on the timestamping threads cuts the negative rate 7–$80\times$ at unchanged utilization. Second, the benchmark keeps an end-to-end sample only when its millisecond-resolution latency is strictly positive and does not count what it discards: of 75 embedded-mode configurations, 71 reported a median that was an exact multiple of one millisecond, computed from 0.36–100% of the samples taken. We contribute an exact identity for the negative span, a law for the retained fraction confirmed by a pre-registered manipulation, and an audit of ten tools. The remedy is to randomize the send instant and publish the retained fraction.**

***Index Terms*—distributed systems; measurement techniques; measurement, evaluation, modeling, simulation of multiple-processor systems; message passing; performance attributes; scheduling and task partitioning.**

I. Introduction

MESSAGE-broker benchmarks inform purchasing and architecture decisions, and the paths they compare now complete in well under a millisecond. This paper reports two ways such a benchmark misreports on those paths without any sign of it in its output. It then says what a benchmark must publish for a reader to tell.

A latency benchmark measures a message by reading a clock when the message is sent and again when it arrives. Subtracting the two timestamps gives the measured latency. The message does not take those readings (Fig. 1); two *threads* do (Fig. 2), each of which must be scheduled onto a core before it can read the clock, and each of which waits for that core on its own account. That wait is the *scheduling delay*: the time a thread that is ready to run spends queued before a core runs it. On a busy machine either wait can exceed the delivery being measured. When the two waits differ by more than the delivery, the subtraction is negative. Nothing arrived before it was sent; the two readings were taken in the wrong order. Separately, the benchmark that the cross-broker comparison literature shares (Section III-A) records its cross-process timestamps in whole milliseconds and keeps only positive differences, so on a path faster than a millisecond it discards samples at a rate fixed by arithmetic rather than chance. Neither failure is visible in the benchmark's output, and both are caught by checks that cost nothing.

Both failures arise when the delivery being measured approaches the timescales of the measurement mechanism. Scheduling delay is a distribution and timestamp resolution is a fixed quantity, and they act differently. Sub-millisecond broker paths are where each becomes comparable to the delivery for tooling in current use. Where that happens is not where a specification says. The timestamp resolution is one millisecond, but the delay that binds is the scheduler's: the last mode of the run-queue stall distribution sits at 3 ms (Section VI-E), three times the timestamp resolution and nothing a specification discloses.

A. Contributions

We make three contributions, each established by manipulation rather than by observation alone.

1) **A law for what a positivity filter discards** (Section V). The fraction of samples a millisecond-resolution benchmark retains is fixed by the ratio of its send interval to its timestamp resolution. It is never shown to the operator. A pre-registered manipulation of message size moves a configuration onto and off the values the law predicts.
2) **An exact account of the negative latency, on one clock** (Section VI). The acknowledgment-referenced span is the delivery less the acknowledgment lag, $S = D - A$ per event. A negative value is therefore an ordering between two measured quantities and not a fault in either clock. Four manipulations separate scheduling from its rival explanations, and the send-referenced span over the same $738{,}730$ events never inverts.
3) **A per-run audit and the reporting rules it implies** (Sections IV-E and VII-B). A sign check applied uniformly to all $2{,}266$ runs rejected $1{,}321$, including every run behind our own first result. Each part of the practice exists already [1], [2], [3]. What we add is their conjunction: the rejection rate published as a headline quantity, and the rule that the retained fraction of samples and the sign of each discard belong beside every reported latency.

G. P. Ricou and D. Gregg are with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie; david.gregg@tcd.ie.

R. Duvignau is with the Department of Computer Science and Engineering, Chalmers University of Technology, Gothenburg, Sweden. E-mail: duvignau@chalmers.se.

N. Herbst is with the Chair of Computer Science II, University of Würzburg, Würzburg, Germany. E-mail: nikolas.herbst@uni-wuerzburg.de.

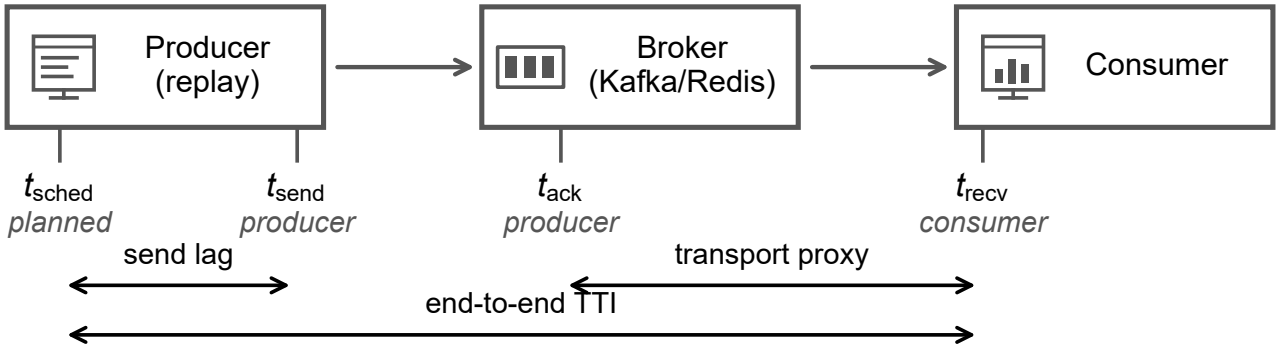

Fig. 1. **The system measured, and where its clocks are read.** A replay process paces recorded match events at a chosen rate into a broker — Kafka or Redis Streams — and a consumer reads them back. The four timestamps and the three spans they bracket are defined in Section II; the transport proxy, the span this paper is about, subtracts a producer-process timestamp from a consumer-process one. Both processes run on one host reading one clock unless stated otherwise, so nothing that follows depends on clock synchronization.

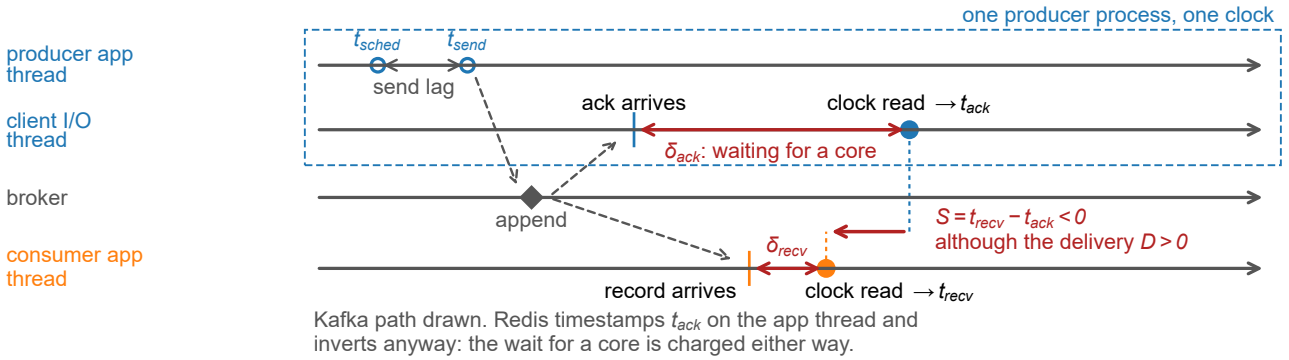

Fig. 2. **A late timestamp, not an early record.** How a positive latency is measured as negative, on one clock. Two threads of the producer process take the two timestamps — the application thread calls send, the client's I/O thread runs the delivery callback — and whichever one timestamps must first be scheduled, so the difference inverts when the acknowledgment's timestamping delay (δ_{ack} in the drawing; δ_{recv} is the consumer's) exceeds the delivery. The path drawn is Kafka's; Redis has no second thread and inverts on a larger share of runs (Supplement S40.3). The producer's own timestamps, t_{sched} and t_{send}, precede the acknowledgment, and the span referenced to them never inverts (Table I).

II. System and Measurement Model

A producer process sends recorded events at a paced rate to a broker, and a consumer process reads them back; Figure 1 draws that path and the timestamps taken on it. Four timestamps bracket a message's path: t_{sched}, when the event was due to be sent; t_{send}, immediately before the send call; t_{ack}, when the producer learns that the broker accepted the message; and t_{recv}, when the consumer holds the record. A fifth, t_{out}, is written by the consumer once it has parsed the record. The difference $t_{\text{out}} - t_{\text{recv}}$ is the consumer's own handling.

Three quantities are defined from these timestamps and used throughout:

$$D \equiv t_{\text{recv}} - t_{\text{send}}, \qquad A \equiv t_{\text{ack}} - t_{\text{send}}, \\ S \equiv t_{\text{recv}} - t_{\text{ack}} \;=\; D - A. \tag{1}$$

D is the *delivery*, from the send call to the consumer holding the record. A is the *acknowledgment lag*, from the send call to the producer observing the broker's acknowledgment. S is the span the benchmark literature reports as broker latency. We call it the *transport proxy*, for the reason given in Section II-A.

The primary end-to-end measure is **Time-to-Insight** (TTI), from scheduled emission to availability at the consumer:

$$\text{TTI} = \underbrace{(t_{\text{send}} - t_{\text{sched}})}_{\text{send lag}} + \underbrace{A}_{\text{acknowledgment lag}} + \underbrace{S}_{\text{transport proxy}}, \tag{2}$$

which telescopes to $t_{\text{recv}} - t_{\text{sched}}$ (Supplement S36 maps every metric of the corpus onto the span it covers). The first term, the *send lag*, is how late the send call ran after the event was due, measured on the producer's own thread. A *span* is any difference of two of these timestamps, and a *negative span* is one whose difference is below zero. The term is Sharma et al.'s [4], kept for the observable while declining the causal reading they give it. It is not RFC 1242's negative latency, a cut-through forwarding artifact. *Retention* is the fraction of the samples a measurement tool takes that survive into what it reports, not Kafka's log-retention setting.

A. The transport proxy is not a causal chain

This distinction decides everything that follows. D and TTI are genuine chains: the producer sends, the broker appends,

the consumer receives, and each event precedes the next. S is not. It subtracts the producer's receipt of the broker's acknowledgment from the consumer's receipt of the record. The broker's append precedes both of those events, but neither of them precedes the other: they are two branches of one cause. The acknowledgment timestamp is therefore a *late, producer-side observation* of a broker-side event, and S is a proxy for the delivery rather than a measurement of it.

Since Equation 1 holds exactly per event, with D and A both non-negative,

$$S < 0 \quad \text{if and only if} \quad A > D, \tag{3}$$

so the span is negative exactly when the acknowledgment lag exceeds the delivery. That is an identity, not a model: both sides count $62{,}264$ negatives over the $738{,}730$ events of our corpus, and the build fails if they part. Every timestamp is written some time after the event it marks, because the thread that reads the clock must first be scheduled (Fig. 2). This is not the cost of the clock call, which is small and separately measurable [5], but the delay before the call runs at all. Only the difference of two such delays matters. When the acknowledgment branch is delayed by more than the delivery, S is negative with nothing physically impossible having occurred.

What a negative S establishes is precise and limited. It does not show that either timestamp is wrong: t_{ack} may record exactly when the producer observed the acknowledgment. It shows that the acknowledgment cannot serve as the origin of a non-negative delivery latency for that event. That is the use the benchmark literature puts it to. The negative-span rate is therefore a property of the quantity being measured, not only of the machine. A fixed distribution of A overlaps a small D almost entirely and a large one hardly at all. That is why a configuration whose transport runs to tens of seconds passes every run while the same hardware measuring one millisecond fails wholesale.

III. Background and Related Work

That measurement error can dominate a benchmark is established; what prior work does not supply is a check applied to every run, and a count of what the check removes.

A. Streaming benchmarks

Broker comparisons appear continuously, in peer-reviewed venues and in the gray literature. Apache Kafka and Redis Streams are the pair they most often name, and the OpenMessaging Benchmark [6] is the tool they share. That experimental setup can dominate a result is established [7], and how to report one is a settled literature we follow [8], [9]. What that literature does not do is check what its tool discards. Most published comparisons report a median above the tool's one-millisecond timestamp resolution, where the deletion law of Section V does not bind. Not all do. Of the two 2026 broker comparisons we placed against that resolution, two report figures at or below it, and one names a percentile for a single broker and none for the other two [10], so its ranking cannot be placed against the resolution at all.

The tools are few and shared, so a defect in one is not one project's problem. The audit of Section V-D reads ten tools at source and finds five disposing of inverted samples without counting them. Coarse timestamp resolution reaches further than the disposal does: the end-to-end tool bundled with Apache Kafka itself computes its percentiles from millisecond-truncated integers [11]. The architecture that produces those spans is not a blunder to be designed away: a producer that blocked for each acknowledgment could not generate load at all. What a benchmark can drop is the habit of subtracting two such timestamps without checking the sign of the result.

That literature already stands its clocks carefully: Karimov et al. [12] separate event-time from processing-time latency, and Van Dongen and Van den Poel [13] use single-broker append timestamps "to ensure correct latency measurements" — our one-clock rule, arrived at independently. Neither asks what a millisecond timestamp does to a sub-millisecond path, nor checks which samples the tool keeps; those two questions are this paper.

B. Cross-process time measurement

Lamport [14] established that a distributed system has no single physical time, only a causal partial order. That order decides this paper's central span. An acknowledgment at the producer and a record at the consumer are two branches from the broker's append, and neither precedes the other. Measurement systems defeat clock disagreement [15] by calibrating cross-node counters [16], timestamping in the NIC [17] or the switch dataplane [18], and measuring the timer's own accuracy [5].

One prior result bounds our claim closely. Villain et al. [19], profiling FreeBSD against a DAG hardware reference, found the outgoing socket timestamp "does not respect causality" — packets leave before the timestamp meant to mark their departure is written — under every stress pattern including none. They attribute host latency to interrupt processing, scheduling and context switching. That is the scheduling mechanism studied in Section VI, published in 2012 and measured more precisely than we measure it. What it does not give, and we add, is its consequence for a two-process measurement: a law relating the negative-span rate to the delivery being measured (Equation 6, Section VI-B), and manipulations that separate scheduling from its rival explanations. The audit half has its own ancestor. Paxson [20] treated physically impossible transit times as measurement failure and published the proportion of traces affected. Lancet [17], a latency-measurement tool that timestamps in the network interface, is the nearest existing gate: it tests each run's samples for statistical stability and reports an inconclusive verdict when they fail, where we decline to publish at all. Neither reports the fraction of runs a campaign lost.

Production tracing reached the same answer without a mechanism: skew adjusters that rewrite a span's timestamps are called harmful and have defaulted to off since 2020 [21], and nobody had measured how much such a correction moves.

Swami and Chougule [22] state the wider position concurrently, that timing observability is itself an interference-

sensitive resource; their reference is a logic analyzer outside the system, ours the sign of a subtraction inside it.

Sharma et al. [4] reach the same observable by another route: they count negative timing spans in distributed inference systems and propose a `causality_health` check. Injecting skew, they see no violations up to 3 ms and clear ones by 5 ms, and read that as a skew threshold. We take their term, and dispute the reading rather than their measurements. They hold that "queueing alone cannot produce negative timing spans". For the queue they model, backlog at a service stage, we agree. A *second* queue decides ours: the run queue the timestamping thread waits in for a CPU. We see rates near 23% at 88% utilization on one host, with an inter-host offset of 0.067 ms where two are used. We move that rate fiftyfold without touching a clock. A threshold in milliseconds of skew does not transfer to a loaded machine. The same attribution was offered for the benchmark we audit: asked why publish latency could exceed end-to-end latency, a maintainer cited inter-node skew of "~100ms" [23] and, told both processes ran on one node, called the measurement reliable. That is the configuration our audit rejects most often.

C. Resolution and discarded samples

That a timer coarser than the interval it measures invalidates the measurement is textbook [5], and older than every tool we audit. Metrology has long carried both our failure modes in one uncertainty budget: the NIST stopwatch guide budgets reaction time beside display resolution, and notes that at short intervals the resolution becomes significant against the stopwatch's rated accuracy [24]. The observation that periodic sampling commensurate with a periodic phenomenon sees only part of it is also old: it is the IPPM framework [25], and RFC 3432 makes a randomized start mandatory for periodic streams [26]. Coordinated omission [27] is the mirror image — there the measurement fails to record samples that would have been slow, here it deletes samples that were fast. Millsap and Holt draw the opposite conclusion from the same arithmetic [28]: coarse timing survives in aggregate *because* the two signed errors cancel. A filter admitting only positive samples removes one sign, and with it the cancellation that defence depends on.

The mathematics is older still and is not ours: Hewlett-Packard's Application Note 162-1 [29] carries our retention identity, the reduced fraction our grid law turns on, our ceiling on resolution gain, the symptom and the cure. What is new is the sign of the operation. HP *keeps* both readings and the retained fraction *is* the estimator; the benchmark discards that population. Our contribution is not the law — we derived it, then found it in a counter manual — but its consequence under deletion. A harness released in 2018 throws away the quantity a 1970 counter measured with.

a) Sources of scheduling delay: A runnable thread does not always run, and work-conservation violations are documented for CFS-era schedulers [30], ours being EEVDF-era. Queueing gives the leading-order account, and the core-count contrast of Table II refutes a single-parameter law rather than queueing itself. Chandrasekar and Kramberger [31] apply such a law to benchmark bias and reach our inflation half independently. We refute it on the load axis (Section VI). What a positive number cannot show remains ours: the negative span, and the sign channel that makes the displacement legible.

IV. Experimental Setup

Everything below is measured on the system of Section II, on one clock unless the text says otherwise. The rule that decides which runs count was fixed before any result was read.

A. Testbeds

Every finding comes from four Oracle Cloud `VM.Standard.E5.Flex` instances, three brokers and one driver. They run Ubuntu 22.04 on kernel `6.8.0-1057-oracle` (EEVDF scheduler) and CPython 3.10, on a private network. Client libraries are pinned identically, and `chrony`-disciplined clocks are logged every minute. The producer and the consumer run as separate *processes on one host*. One early phase perturbed the measurement enough to be unusable and is excluded from every result here. The campaigns that remain measure utilization with no core pinning (Supplement S1). A second testbed, a workstation running the brokers under containers, produced our withdrawn first result and appears only as an audit outcome.

B. Terms

A *condition* is one point of the experimental design: a broker, a send rate, a message size, a background load and a consumer pattern. A *cell* is a condition together with a workload; where the workload is fixed the two coincide. The benchmark audit of Section V counts cells because it varies the workload. A *run* is one execution of a cell, and a *replicate* is a further run of the same cell. Where a manipulation compares settings that differ in one variable, each setting is a *configuration* of that manipulation. Utilization ρ is the fraction of CPU time busy on the host, measured over the run rather than configured. The core count k is the number of cores the background load occupies; both are manipulated in Section VI. Two settings that reach the same utilization with the background load on different numbers of cores differ in *load geometry*: *concentrated* on few cores or *spread* over many.

C. Clocks

Both processes read the wall clock (`time.time_ns()`, CLOCK_REALTIME). The two-host configurations need a common epoch, and the choice is kept on one host so that every configuration is measured the same way. On one host CLOCK_MONOTONIC would have guaranteed that consecutive reads never go backwards; we forgo that exclusion and establish it from data instead (Section VI; Supplement S39). The Time Stamp Counter (TSC) is a per-core cycle register and a separate matter: whether the kernel's clocksource is coherent across virtual CPUs is bounded in Section VII-D. The measured inter-host offset, where two hosts are used, is 0.067 ms.

D. Workload and load generation

The workload is StatsBomb's open football event data [32] (CC BY-NC-SA 4.0): per-match event streams — passes, shots, pressure events — with sub-second event times. We replay 3,315 matches, preserving each match's event order and scaling its inter-event spacing to the chosen send rate. It stands for small frequent ordered messages rather than for any particular deployment, and nothing below depends on its content beyond message size and rate (Supplement S29). The send schedule is measured rather than assumed. Pacer jitter is 67–69 μs at p90, and the achieved rate is recorded per run against elapsed wall time: the self-validation SPEC and TPC ask of a benchmark [3], applied to the load generator.

The two clients place one timestamp differently, and it matters to every span referenced to t_{recv}. Kafka's client deserializes the payload before that timestamp and Redis's after it, so such a span carries one client's payload parse and not the other's (Supplement S40.1); Section VII-A says what that is worth.

E. The consistency check

We fixed the rule before the final campaign and apply it to every run, including those whose results looked reasonable. **A run is rejected if more than 1% of its events carry a negative value in any latency component, or if the median of any component is negative.** A condition is usable only if all of its runs survive. Where a condition is partly rejected we report the fraction of its runs that survived. The one-per-cent figure is a threshold, not a discovery. The supplement sweeps it from 0 to 20%, and no condition behind our first result is usable anywhere in that range, so the withdrawal does not turn on where the line was drawn.

The justification is not that a negative is impossible, because for the transport proxy it is not (Section II-A). It is that a negative span shows the reference timestamp has failed as an origin for that event. A run whose reference is unusable on more than one event in a hundred cannot report a latency, whatever the cause. The check is a *necessary* condition, not a validation procedure. A corpus that fails is unusable, and one that passes has cleared the cheapest available bar and nothing more. We use the sign as a gate rather than as an estimator. The same constraint supports estimation, since offset and skew can be recovered from a delay envelope [20]. An estimator corrects a record. A gate declines to publish from a run whose measurement demonstrably failed, and after this check destroyed our own first result we preferred declining.

F. Statistics

Every proportion below is a count over a stated denominator with a Wilson score 95% interval, not Wald: at the real-time configurations' rates the normal approximation is shifted low and covers below its nominal rate. Holm correction is applied within a named family, and the corrected value is the one the tables display. Tail indices are estimated on the traced histogram by grouped maximum likelihood and by an exceedance ratio. A slope through survival points would mislead, because the buckets are interval-censored. The grouped estimator on log-spaced bins is Virkar and Clauset's, the Hill estimator for binned data, and we use their bootstrap for goodness of fit [33]. The remaining estimators are inventoried in Supplement S35. All interval arithmetic is recomputed from the committed artifacts at build time by `scripts/stat_intervals.py` and `scripts/tail_index_traced.py`. The tables reporting it are generated rather than transcribed.

V. Quantization and Silent Deletion

The cross-broker benchmark we audit computes its reported latency distribution from a subset of the samples it collected, does not record how large that subset was, and lets the send rate decide it. Everything measured below is the Open-Messaging Benchmark in embedded mode, where driver and workers run in one JVM. Its distributed mode is discussed in Section VII-D.

A. The admission condition, and its origin

In `LocalWorker.java`:294 the end-to-end latency is `now - publishTimestamp`, where `now` is read in the consumer and `publishTimestamp` is Kafka's producer-set `CreateTime`, itself a millisecond value. That difference is then promoted to microseconds (Supplement S37 quotes the call). A sample then enters the histogram only if it passes the test in `WorkerStats.java`:95 — `if (endToEndLatencyMicros > 0)` — which we call the *admission condition*. The message is counted as received a few lines earlier, but a non-positive latency is dropped and no counter records how many. Over the campaign of Section V-B that silence covers 11,532,492 samples.

The condition's origin is documented and its reasoning is sound. It was added in 2018 to stop negative values reaching the histogram, on the ground that end-to-end latency "is calculated based on time difference of `System.currentTimeMillis()` on two different machines" and "can be negative" [34]. The recorder is an `HdrHistogram Recorder`, which rejects negative values outright, so filtering for positives is the reasonable local fix, defensive rather than evasive. Its non-local consequences are two, neither considered at the time. First, `> 0` also deletes every sample that computes to exactly zero. Because `System.currentTimeMillis()` has millisecond resolution, any delivery faster than one millisecond does compute to zero. On co-located paths, where our own transport measures 0.1–0.5 ms, that is most of them. Second, the one observation which would show the measurement had failed is the exact observation the condition removes. Section V-D finds the same disposal, uncounted, in four independent tools.

The condition's scope is telling. It is on the end-to-end path, which timestamps in milliseconds; the same harness measures its publish span with a nanosecond clock and records every sample unconditionally. The coarse clock and the filter both landed on the one span that crosses processes.

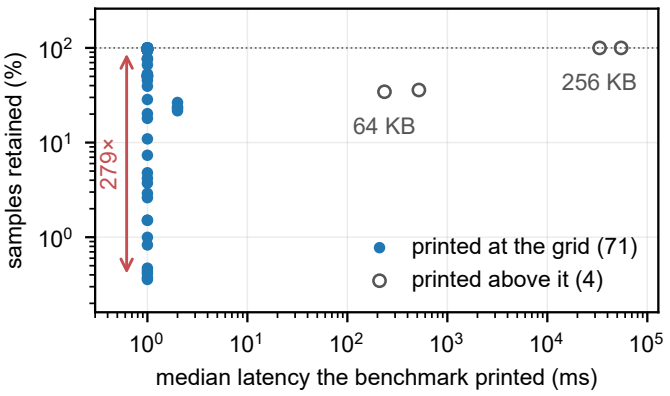

Fig. 3. **What the benchmark prints against what it kept in order to print it**, over every instrumented cell. At the timestamp resolution the printed median carries no information about retention: 71 cells report 1.0 or 2.0 ms while the retained fraction spans 279×. Away from it the harness behaves: the failure is confined to deliveries shorter than the timestamp resolution.

a) The consequence, measured: Of the 75 instrumented cells whose own summary we captured, 71 report a median of 1.0 or 2.0 ms, and across those the benchmark computes its reported distribution from between $\mathbf{0.36}\%$ and $\mathbf{100}\%$ of the samples it takes. Two replicates of one cell — 200 B payload at 500 msg/s, same host — kept 431 and 120,434 samples, a 279-fold difference, and both reported a median of 1.0 ms with nothing in the output to distinguish them. Three uninstrumented runs print p50, p95, p99 and max all equal to 1.0 to three decimal places. The full ledger holds 223 instrumented runs, 11,532,492 discarded samples and 41,403 negatives, and over it the retained fraction falls as low as 0.0044% — four samples kept of ninety thousand. The printed median is pinned at the timestamp resolution while retention moves across two and a half orders of magnitude (Fig. 3). The only cells that escape are the four whose payload is large enough that the resolution stops binding.

That discarded fraction is the one time-interval averaging measures with [29]: reporting a distribution conditioned on it, without reporting it, keeps the residue and throws away the estimator.

B. Phase, and the grid it produces

Publish latency across these cells takes only two values, 0.3 and 0.4 ms, while retention varies 279-fold. A two-valued predictor cannot account for a range like that whatever its correlation, and the correlation it does have (+0.31, Fisher +0.08–+0.51, over 71 cells) carries the sign Equation 4 predicts. What moves retention is where the producer's send instants fall against the *grid*: the instants, one millisecond apart, at which the timestamp changes value. Write τ for the timestamp resolution — the *quantum*, one millisecond here — and T_{true} for the true delivery of a message, the interval D would report if it were measured with an arbitrarily fine timestamp. A sample survives when its delivery crosses a grid instant, which for $T_{\text{true}} < \tau$ happens with probability T_{true}/τ (Supplement S42 draws the geometry):

$$\text{retention} \;=\; \min\left(1,\; \frac{T_{\text{true}}}{\tau}\right) \qquad \text{(uniform phases).} \tag{4}$$

All seven configurations whose send interval is incommensurate with τ sit near 50% — medians 47.2 to 51.5 — implying $T_{\text{true}} \approx 0.5$ ms at $\tau = 1$ ms.

a) Commensurability is not binary: Phases are not uniform when the send interval is commensurate with the tick. Write the send interval over the tick in lowest terms as $\Delta/\tau = p/q$ with $\gcd(p, q) = 1$. A producer then visits exactly q distinct phases. Retention is a count out of q and can take only the $q{+}1$ values $0, 1/q, \ldots, 1$, the *vertices* of its grid. A run lands on one of the two vertices bracketing T_{true}/τ. The replicate spread follows:

$$\text{spread} \;\longrightarrow\; \begin{cases} 100/q & \text{when } T_{\text{true}}/\tau \text{ sits mid-cell,} \\ 0 & \text{when } T_{\text{true}}/\tau \text{ sits on a grid point.} \end{cases} \tag{5}$$

Both branches are limits: a nominally commensurate rate is not exactly commensurate, and the two configurations that sit on a vertex spread by roughly three tenths of the mid-cell width, too few to fix a constant (Supplement S19). A round send rate is the worst possible choice, and exactness decides it: Supplement S19 runs from the $q = 1$ rate whose replicates span almost the whole range to the one sitting 0.00014 from a vertex and behaving as fully continuous.

C. The grid as inference

A table of spreads is not a test, so we state one: the mean normalized distance from a configuration's replicates to the nearest q-grid vertex, against the within-configuration permutation null that a *continuum* — retention varying smoothly with T_{true}/τ, as an incommensurate rate gives — implies. Supplements S44 and S23 give the construction and every configuration with $n \geq 5$. 9 of the 10 powered configurations reject the continuum null at $\alpha = 0.05$ after Holm correction within the family of 12. The tenth, 900 msg/s, rejects before correction ($p = 0.044$) and not after ($p = 0.131$); we report it as unresolved rather than as support. The remaining 2 configurations, 400 and 800 msg/s, have no power by construction: where T_{true}/τ sits on a vertex the two predictions coincide, so the distance is replicate noise and which side of the diagonal it falls on is not evidence either way (Fig. 4).

a) A pre-registered confirmation: Payload moves T_{true}, hence the grid position, and Equation 5 dictates the configuration's class with nothing else changed. The prediction was registered before the runs, and it came true: spread collapses to 13.6 at 32 KB, pinned near the $2/3$ vertex, and re-opens to 26.7 at 64 KB — the flat–full boundary crossed in both directions by one manipulated variable. One registered detail failed — the 32 KB pin sits at 69.2, not the predicted 66.67. Supplement S22 draws all three configurations against their grid,

and a larger campaign leaves per-send jitter rather than accumulation standing (Supplement S26).

D. Generality, and the sign channel

To show the law is not an artifact of one JVM, we reproduced the admission condition in an independent Python harness sharing no code with the benchmark. On one clock

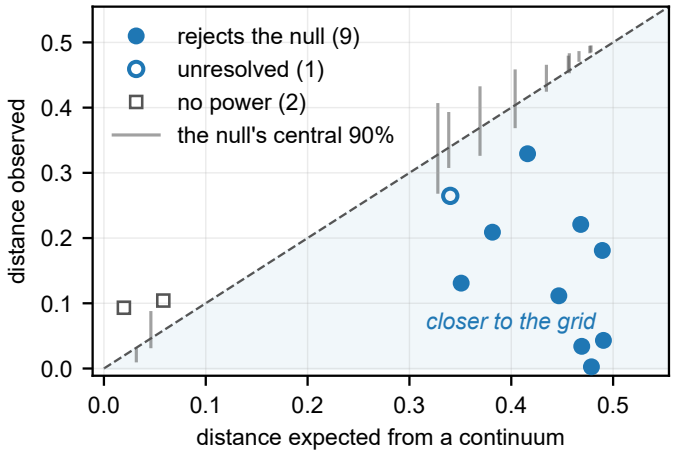

Fig. 4. **Grid membership, as a set.** Each configuration's observed mean distance to the nearest grid vertex against the distance a continuum would give, which is the diagonal. The gray bar beside each configuration is its own simulated null, *uncorrected*; the test is one-sided towards the grid, so a marker *below* its bar rejects that null. Filled circles also survive Holm correction. The open circle sits *below* its bar too, 0.2647 against a band opening at 0.2680, but at $p = 0.044$ raw and 0.131 corrected it is the one powered configuration that does not reject: correction is what the picture cannot draw. The 2 squares are the configurations where the two hypotheses coincide and no test has power; *the sign carries nothing* there. Corrected p-values are in Supplement S23.

it recorded **zero** negative differences across 905,040 samples, as a causal chain requires, and none across two hosts either (Supplement S11). The grid behavior reappeared intact. At 457 msg/s — the incommensurate configuration, the only one of the three whose own grid spread does not swamp the comparison — cross-host retention wandered from 13.4 to 26.9% over four replicates, where the same configuration on one clock held to 0.98 points. Cross-host, the deleted fraction couples to the synchronization state and nothing records either. Nor is the admission condition confined to the tool we measured: 40 of the 40 public forks we could read carry the expression unchanged, surveyed 2026-08-24.

Nor is the disposal confined to this harness, and deletion is not the only way coarse resolution wins. We audited ten tools at source across five languages and nine vendors, classifying each finding with the code that classified the benchmark. Five of the ten dispose of the sample without counting it, in three classes — a pattern rather than one project's oversight, and one of them is Apache Pulsar's performance client reaching the same design independently [35]. Apache Kafka's own bundled end-to-end tool keeps every sample and loses it anyway, filling the array its percentiles come from by integer division, so on a sub-millisecond path every one reports zero [11]. Two more keep their negatives and pass them on unfiltered, as RFC 7679 asks [36], [37]. The failure is not universal, and the claim that survives is the narrower one: *the tools that quantize or filter do so without counting*. Substitution is the worst of the three classes because it leaves nothing missing: `fio` and `btt` substitute zero and one nanosecond, and a value never measured enters the distribution. At the other end, one counts its discards in a metric whose description names the cause [38], which is what makes the rule of Section VII-B practical rather than aspirational. Supplement S37 gives the registry tool by tool, with the source line each finding sits on and one entry that corrects a misreading of our own. The class is not confined to benchmarks: in November 2025 the audited broker's own coordinator metrics adopted the same disposal, clamping negative durations to zero with no counter [39] (Supplement S37).

Separating the discards by sign is what makes the condition's cost legible. Across the Kafka-driver corpus's 214 runs its 10,913,263 discarded samples contain **not one negative**: the most negative value at any load, size or rate is 0 μs. Those discards are resolution failures, not clock failures — a distinction an unsigned counter cannot make. The same channel then caught the real thing in the Redis-driver replication's 9 runs: 41,403 negatives, every one exactly $-1000\,\mu$s, the smallest value a one-millisecond timestamp can express, with 41,397 of them in a single run where they were *half of all samples taken*, beside six kept.

That value is not a coincidence. The Redis driver's span is a difference between two millisecond-floored clocks on two hosts, and a sub-tick offset between two floored clocks can produce exactly one negative value, $-1000\,\mu$s, and no other. That is precisely what the corpus contains. The Kafka-driver span, floored the same way but taken inside one JVM, never inverts at all. Clock discipline was clean at every logged minute, which is the point — this failure needs no misbehaving clock, only two of them and a floor (Supplement S43).

VI. Scheduling and Acknowledgment Bias

The negatives in our own corpus are produced by scheduling, and we establish this by moving scheduling while holding everything else fixed.

A. What the audit rejected

The rule of Section IV-E rejects 862 of 1,382 runs on the workstation testbed (62.4%) and 459 of 884 on the cloud testbed (51.9%), every run behind our first result among them: on that first testbed 8 of 76 conditions survive.

Table I separates the components over a wider population than the audit: every cloud run that produced matched events, 5,913 of them, rather than the 884 behind a reported result. A condemned run answers which span inverts as well as a clean one does, so none is excluded. The acknowledgment-referenced proxy is negative on 62,264 of 738,730 events (8.43%; Wilson intervals are under 0.1 points here and omitted hereafter). Of the runs, 3,976 exceed the one-per-cent threshold on it. The send-referenced span and TTI, which are causal chains, are negative on **no event at all**. None of the 62,264 messages arrived before it was sent; the acknowledgment timestamp was written late. The rejections are therefore evidence that the acknowledgment cannot serve as the origin of a delivery latency on those events, which is precisely what the check of Section IV-E is for.

The sign is a threshold, so the negatives count only the worst of the displacement. The acknowledgment lag exceeds half the delivery time on 74.5% of events and a tenth of it on 95.2%. The check fires only on the 8.4% where it exceeds the whole. The two are correlated within a run (median 0.84 over

TABLE I
NEGATIVES BY SPAN AND BROKER (5,913 RUNS, 738,730 EVENTS, ONE CLOCK). ONLY THE ACKNOWLEDGMENT-ORIGIN SPAN INVERTS, AND IT DOES SO UNDER BOTH BROKERS: KAFKA ON 8.67% OF EVENTS, REDIS ON 8.19%. THE CAUSAL CHAINS ARE CLEAN UNDER EACH BROKER, SO THEIR PER-BROKER COLUMNS ARE DASHED.

Span	Origin timestamp	Kafka	Redis	All
$t_{\mathrm{recv}} - t_{\mathrm{ack}}$	acknowledgment	31,899	30,365	62,264
$t_{\mathrm{recv}} - t_{\mathrm{send}}$	send (chain)	—	—	0
$t_{\mathrm{out}} - t_{\mathrm{send}}$	send (chain)	—	—	0
TTI	emission	—	—	0

70 conditions), one scheduler causing both, which suppresses crossings without touching the displacement. A path longer than the timestamp resolution is not clean but quiet.

The rejected result had looked correct: monotone in concurrency, significant after correction ($p = 9.0 \times 10^{-11}$), and past every conventional check we knew (Supplement S40). The check does not catch everything. Load-induced inflation that stays positive is physically possible, and our own second withdrawal is the example — a twentyfold end-to-end gap that was a per-run start-up cost read as a per-event constant, causally consistent and wrong. Causal consistency is necessary, not sufficient.

B. A rare state, not a wide distribution

Each timestamping thread is either running, in which case its timestamp is late by a narrow jitter of width σ_c, or preempted and off-CPU. Write p for the probability of the second state — the timestamping thread's off-CPU *occupancy*, which is what Table II moves — and $G(\cdot)$ for the survival function of the residual delay in that state, and take the consumer's timestamp to lie in the core. Write T_{true} for the true delivery of Section V-B: the quantity D measures, equal to D up to the consumer's own timestamping jitter. Conditioning on it, the identity of Equation 3 becomes, to leading order,

$$\Pr[S < 0 \mid T_{\mathrm{true}}] \;=\; p\, G(T_{\mathrm{true}}). \tag{6}$$

Load moves p and does not fix it — two geometries at one ρ, rates twofold apart (Table II, lower panel) — so we write p and not $p(\rho)$, and the model's content is on the delivery axis rather than the load one (Supplement S13). The leading-order step does not assume the two states are independent: a stall delaying both branches cancels in $S = D - A$, so what the model sets aside is invisible rather than rare. Load changes how *often* the thread that must record the time is not running, not how wide the ordinary jitter is. Going from idle to the knee multiplies the fitted core width σ_c by 5 and the negative-span rate — the mass of A beyond D — by 61. The rate has a ceiling below one: fully saturated it reaches 0.37, so even then most timestamps are taken by a thread that was running, or late by less than the delivery.

C. The experiments that settle it

No pair of configurations in Table II overlaps, and Supplement S24 draws them. Real-time priority on the timestamping

TABLE II
THE MECHANISM, BY MANIPULATION (2,985 MATCHED EVENTS PER CELL). TOP: TIMESTAMPING THREADS MOVED TO `SCHED_FIFO` WITH BACKGROUND LOAD UNCHANGED; ACHIEVED UTILIZATION MATCHES TO WITHIN 0.0006 BETWEEN CONFIGURATIONS, SO OCCUPANCY ALONE MOVED. BOTTOM: IDENTICAL UTILIZATION REACHED BY TWO CORE GEOMETRIES ($\rho = 0.7531$ IN ALL FOUR CONFIGURATIONS WITH k=6 LOADED CORES); A FUNCTION OF ρ RETURNS ONE VALUE FOR ONE INPUT.

Manipulation	Configuration	Rate	95% CI	Factor (z)
Priority, 75%	ordinary	0.1320	0.1203–0.1446	39× (19.8)
	real-time	0.0034	0.0018–0.0062	
Priority, 88%	ordinary	0.3049	0.2886–0.3216	54× (31.9)
	real-time	0.0057	0.0036–0.0091	
Geometry, original	concentrated	0.0824	0.0731–0.0928	2.07× (10.3)
	spread	0.1709	0.1578–0.1848	
Geometry, repl.	concentrated	0.0600	0.0520–0.0691	2.05× (8.4)
	spread	0.1229	0.1117–0.1352	

threads moves occupancy while utilization stays fixed (Table II, top). The rate falls by factors of 39 [21–74] and 54 [33–86]. The brackets are Katz intervals on the ratio, wide because the collapsed configurations are sparse; the Wilson intervals are disjoint at both load levels ($z = 19.8$ and $z = 31.9$). Both residual rates land on the scale the model names in advance. That scale is the floor C_0: the negative-span rate that remains when the timestamping thread is never preempted and only the core jitter σ_c is left, $C_0 \approx 0.004$. Over all 8 pairs the manipulated configuration runs from 0.0017 to 0.0144, so C_0 is where the rate settles and not a bound it respects. *This refutes every account in which the negative-span rate is a function of utilization*, including the queueing form we ourselves adopted: ρ did not change and the rate changed fiftyfold. Across 8 matched pairs spanning 60 to 95% load the factor ranges 7–80×, every pair with disjoint intervals (Supplement S24).

Three further manipulations agree. At $k = 6$ both load geometries sit at $\rho = 0.7531$, identical to four decimals, and the negative-span rates differ by 2.07× [1.80–2.39] ($z = 10.3$), reproduced at 2.05× [1.73–2.43] by a full replication. Multi-server queueing expects geometry to matter at fixed ρ; what it rules out is the single-parameter form we had adopted and pre-registered, where ρ alone fixes the rate. Payload padding lengthened transport 76.9× while the negative-span rate *fell* 4.1× [3.5–4.8], monotonically, at a 0.012 utilization spread — the direction Equation 3 predicts and the opposite of a stress account. Finally, a kernel trace on the `sched_wakeup` and `sched_switch` tracepoints [40] predicts that rate to within a third across 3 configurations at two load levels — ratios $0.78, 1.06, 1.32$, no consistent sign, nothing fitted. It is filtered to the harness processes and shares no probe, estimator or data with the application-level rate. The probe is not free: 0.272 untraced against 0.231 traced ($z = 3.6$), and the comparison uses the traced configuration's own rate, so 0.272 is the machine's (Supplement S40.4). The knee sweep placed conditions where the candidate load laws diverge and both failed, because both were parameterized in ρ and ρ is not the variable (Supplement S13).

D. What the proxy costs a comparison

The displacement A is a scheduling quantity and does not grow because a message travelled further, so the relative error of the proxy is A/D and falls as $1/D$. At our median acknowledgment lag, $A = 725\,\mu$s, that is 7% at a 10 ms delivery, 1% at 100 ms and 72% at 1 ms; below 0.72 ms the displacement exceeds the delivery it is a correction to, and sub-millisecond medians are routine in broker benchmarking. A is not a constant: across our conditions it spans 500–1900 μs, putting the 10 ms error at 7–19% and the crossover at 0.72–1.90 ms (Supplement S24 bands every row and draws the curve).

Two consequences follow, both measured on the gated corpus. First, two *identical* systems whose clients timestamp in different places are separated by the difference of their lags: a reported gap of 11% at 10 ms, with the sign check firing at neither. Second, on this corpus the reported span is 24% of the delivery it stands for, understating it 4.2× — a median of per-condition ratios, which is not what differencing two published medians gives (Supplement S24). The two brokers' clients timestamp differently, and over 24 matched workloads the proxy inflates the gap between them 1.7× (interquartile range 1.2–2.0), narrowing it on 21% of the pairs: a property of the clients, not of the brokers.

The displacement can be recovered rather than discarded. Equation 1 holds per event and A is producer-side on one clock, so a harness that records its own acknowledgment lag recovers the delivery it failed to measure. The recovered median lands within 3.4% of the true one on runs the check passes, and within 12.2% on the runs it rejects — the population a remedy tried only against clean runs never meets.

E. The stall spectrum, and the scheduler's slice

That this distribution is not a single heavy tail is known: B. Gregg reported it as bimodal in the post introducing `runqlat`, the tool we use [40]. The `timerlat` tracer decomposes scheduling latency into its interrupt and thread components [41]; what we add is which modes, and why the upper one sits where it does.

Log–log least squares over the four payload levels of Section VI-C returns a slope of -0.34 (-0.44 to -0.23); four points do not earn an equation here, and an earlier version read the kernel trace as confirming it. We withdraw the claim, because estimating the traced index properly is what refuted it: two estimators disagree sixfold, and a bootstrap goodness-of-fit rejects the fitted power law ($p < 0.0004$ over 2,500 replicates), both in Supplement S15.

Over the 551,956 traced wakeups the counts do not descend monotonically, and on $\log_2$ buckets a power law of index above one must. What the counts have instead is 3 local maxima. The last is a *mode* at 2–4 ms, holding 10.5% of all wakeups and standing 4.5× its lower neighbor. Above it the survival is light: grouped maximum likelihood beyond 4 ms gives 2.04 (2.00–2.07), a finite variance. Neither that share nor that position has been attributed to a scheduler constant before; Figure 5 is the histogram, and no monotone curve passes through it.

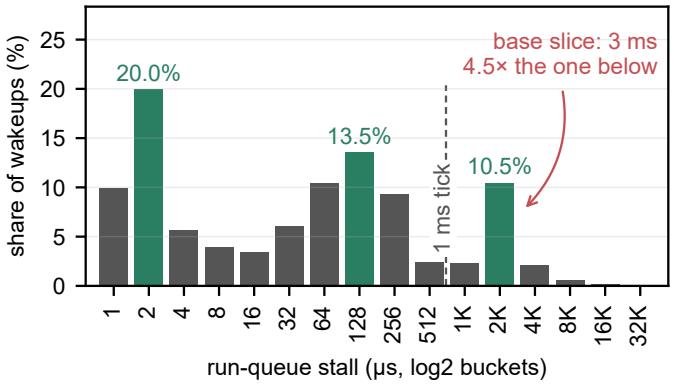

Fig. 5. **The traced run-queue stall distribution is trimodal.** 551,956 wakeups, `bpftrace` $\log_2$ buckets, modes shaded. The jitter core sits at a few microseconds and a second mode at 128–256 μs; the third, holding 10.5% of all wakeups, lands on the scheduler's derived base slice of 3 ms. A power law of index above one — the fitted index is 2.04 — would have to descend monotonically across this axis.

That mode is the preempted state of Equation 6, at a scale the scheduler's own maintainers had already named: with run-to-parity a task's lag is held within "$[-(\text{slice} + \text{tick}) : (\text{slice} + \text{tick})]$" [42]. The histogram adds that this is where a tenth of all wakeups land, not merely a worst case. The slice is *derived* rather than measured: the kernel's published rule [43] gives $4 \times 0.75\,\text{ms} = 3$ ms on the 8 cores the campaign's own load geometry recovers, over a 1 ms scheduler tick, and no scheduler tunable is written anywhere in the archived campaign (Supplement S41). That tick is the kernel's timer period, not the timestamp resolution of Section V; both are 1 ms here by coincidence of configuration. A thread descheduled at the wrong instant therefore reads its clock about a slice late, six to thirty times a 0.1–0.5 ms delivery, and the tick is why the mode is a band rather than a spike. A mean over this distribution is dominated by a mode the operator never sees, so a mean-based counter cannot flag this failure.

VII. Discussion

Both failures are cheap to detect and neither is detected, so the remedy is a reporting rule rather than a better clock.

A. What the brokers actually do

Once the failed runs are removed, the broker comparison that motivated the work is small and one configuration choice dwarfs it. On gated artifacts the two sit within a millisecond on the transport proxy (TOST against a 1 ms margin, $p < 0.001$, across 3 levels), so it is not a purchasing argument. The one condition that reverses the ordering is a consumption pattern rather than the broker, and a co-located testbed — the standard evaluation environment — hides it (Supplement S25).

Our own measurement is not symmetric across that comparison, and the rule below says to declare it. Kafka's client deserializes the payload before t_{recv} and Redis's after it, so the transport proxy carries one client's parse and not the other's: 19.5 μs, or 2.4% of Redis's own median delivery. That is a fiftieth of the equivalence margin above, and it runs *against*

the per-broker difference of Table I rather than producing it. TTI is free of it, because both parses fall inside TTI (Supplement S40.1).

B. For benchmark authors

SPEC and TPC judge a benchmark on five criteria [3]. The tool we audit passes the half of *reproducibility* a reader checks — run-to-run consistency at a fixed configuration — *because* it fails *verifiability*. A median from 0.36% of the samples is the more stable for having had the variation deleted, and a second tester who picks a different send rate measures a different sample. Every rule below instances the criterion that can see this.

Report a physical-consistency audit. Any cross-process timestamp difference admits a sign check that costs nothing, and every run it rejected here had passed every conventional check we knew. Better measurement systems exist [16], [17], [18]; whatever the measurement, check its output against causality and publish the rejection rate, as network clocks have for thirty years [20], [1]. The discipline is tested only by a campaign the rate costs something, and ours cost us a result.

Name the span you are subtracting. A difference of two timestamps is a latency only if the first event causally precedes the second. An acknowledgment at the producer does not precede a record at the consumer (Section II-A). We caught our own instance only on counting the send-referenced span separately (Table I).

Where the span cannot be re-timestamped, recover the displacement rather than discard it. A harness that records its own acknowledgment lag recovers the delivery it failed to measure, on the runs the check rejects as well as on those it passes (Section VI-D). This does not reopen the gate of Section IV-E: that rule declines to publish the proxy as a latency, and this one reports the delivery instead.

Two delays with one cause are not independent. Timestamps waiting on one scheduler move together ($\rho = 0.84$), so multiplying their probabilities misprices the answer. Assuming independence here overpredicts the negative-span rate in all 70 conditions, by a median factor of 12.4.

Know where your own path sits on the exposure curve, and how wide it is. The relative error of an acknowledgment-referenced span is A/D (Section VI-D). Measure your own A: ours spans an order of magnitude across conditions and moves the crossover with it. A harness that subtracts a send timestamp is not on the curve at all, which is the whole return on naming the span.

Count your events before you quote a percentile. A median over as few as seven events per run is not a property of the system, and repetition will not tell you so.

Make the timestamping path unpreemptable, and say that you did. The one mitigation our measurements support directly: real-time priority on the timestamping threads cut the negative-span rate 7–$80\times$ at unchanged background load. Busy-polling a dedicated core should buy the same for the price of that core, though we did not measure it. Two caveats: the floor is not zero, so the check stays. And a real-time or polling timestamping thread changes the system it measures, so where that is unacceptable, report retention instead.

Dither the send instant, and publish the retained fraction. Randomizing probe timing to avoid synchronizing with a periodic phenomenon is long-standing advice [25], [26]; what we add is why — the phenomenon synchronized with is the timestamp resolution itself. Publish beside the latency the retention rate, the timestamp resolution and the synchronization state, none recoverable afterwards. A summary from 0.36% of samples is typographically identical to one from all. Neither half of the rule is ours: OWAMP has attached an error estimate to every timestamp since 2006, and records a packet timestamped in the future unaltered rather than discarding it [44] (Supplements S49 and S50 give the precedents). The sign is the channel: negative means the reference timestamp failed, computes-to-zero means the resolution failed, and one unsigned total cannot distinguish them.

Record the achieved rate, not the flag meant to produce it. Verify it per run against elapsed wall time. The send schedule is an experimental variable in Section V, not a nuisance parameter. We found runs where configured and achieved rates disagreed.

None of this is hypothetical: `mq-bench`, a 2026 framework for the same comparisons, already timestamps in nanoseconds and subtracts a span that cannot invert, both choices made and neither argued for (Supplement S52.2).

C. The limits of a better clock

Hardware-timestamped PTP [15] would take two LAN hosts from our $67\,\mu$s of `chrony` agreement to sub-microsecond. Behind a one-millisecond timestamp, $67\,\mu$s and 70 ns produce byte-identical records. The admission condition deletes the same samples and the grid law is unchanged. The remedies are therefore ordered: record at native resolution, count what is discarded, dither the send instant, and only then is synchronization quality the frontier (Section V-D). Resolution is the lesser half, and a distinct one. A microsecond timestamp retires Equation 4's deletions here and moves the grid to faster paths, but the admission condition and genuine negatives survive it (Supplements S47 and S40.6). A better clock alone therefore converts an uncounted *resolution* failure into an uncounted *timestamping* one.

D. Threats and limitations

A negative span shows that the acknowledgment cannot serve as the origin of a delivery latency for that event; attributing that to scheduling is an inference, separated from its rivals by manipulation, with the eliminations listed below. Beyond that list, the clustering of consecutive late wakes ($z = -6.9$ at the co-located floor, at every load including none) is the signature of a shared scheduling delay, not independent kernel granularity. The `chrony` daemon reports the inter-host offset as comfortably sub-tick but bounds its own error at 4–7 ms per host; the two worst bounds sum to 12 ms against a 1 ms timestamp, so the cross-host channel stays open rather than ruled out. It did not produce the Redis-driver negatives, which the driver's own timestamp accounts for exactly (Section V-D),

but it is why we claim no cross-host bound in general. The skew threshold of Section III-B does not transfer to a loaded machine either (Supplement S51).

a) Scope, and the alternatives eliminated: Three widely used runtimes document this structure without drawing its consequence for a cross-process subtraction (Supplement S48). The mechanism requires a timestamping path that can be preempted. Designs that pin a thread to a dedicated core and busy-poll, or that timestamp in hardware, remove the state Equation 6 depends on, and we make no claim about them. Within our own setting the alternatives are each eliminated. Clock skew: one clock by construction on the rejecting configurations, and 0 negatives in $905{,}040$ two-clock samples. Cross-CPU incoherence under migration: below. Utilization: identical ρ, rates twofold apart. Timer noise: `SCHED_FIFO` at fixed utilization collapses the rate across 8 pairs, and noise does not respect scheduling class. Stress: transport $77\times$ longer *lowers* the rate. Direct observation: a `sched_switch` histogram predicts the rate to within a third, unfitted. One rival is not the scheduler's — the timestamp is a CPython call, so it waits on the interpreter lock too, invisibly to a tracepoint probe. That one is bounded because the kernel-only estimator brackets the observed rate $(0.78, 1.06, 1.32)$ instead of underpredicting (Supplement S40.4).

Clock incoherence across vCPUs, our machines being virtual [45], is closed twice. The probe's 90 ns increment admits only `kvm-clock` or `tsc`, both of which the kernel uses solely where cross-CPU coherence is asserted; and the corpus closes the channel again on headroom, not on shared endpoints, since the send-referenced span holds a floor $455\times$ too high to explain the deepest negative span (Supplement S39).

The mechanism's constants are those of one EEVDF-era kernel (6.8); CFS-era schedulers, the regime Lozi et al. document, may shift them. The deletion law is demonstrated on one benchmark and reproduced in one independent harness. The five tools of Section V-D are readings of source rather than measured deployments. The benchmark's distributed mode failed on every attempt, so its cross-host exposure rests on source reading and our own two-host harness (Supplement S34). Exposure depends on a deployment's path latency and producer rate. That is why we recommend publishing a retention rate rather than calling any published number wrong.

VIII. Conclusion

A latency benchmark measures the system only while its timestamps are taken faster than the delivery it measures. Below that line the two failures reported here appear, invisible in the output. Two timestamps written by threads that wait for a core independently invert the delivery between them whenever those waits differ by more than it. A timestamp quantized to a millisecond, filtered for positivity, deletes most of a submillisecond path by arithmetic the operator never sees. Neither needs a better clock, and neither is expensive to detect. Check the spans whose endpoints are timestamped in different places, count what your tool throws away, and publish the retention rate beside the number.

Acknowledgment

The authors thank J. Kunkel, for the same-clock protocol contrast, the offset-estimation route and the measurement-model figure; M. Luthra, for the kernel-tracing pointers; M. Kogias, for the probe-placement argument; and L. Tratt, for the question about timestamp resolution.

In accordance with IEEE policy, Anthropic's Claude assisted with code development and verification, manuscript editing, and literature search. Experimental design, scientific claims, and interpretation are the authors'.

Artifact Availability

Code, analysis and manuscript are archived at 10.5281/zenodo.21650031, the measurement dataset at 10.5281/zenodo.21650064; both resolve to the current version, `v3.0.0`. Every number here is recomputed from the committed data by the archived code at build time, and the test suite fails if the text and the data disagree.

References

[1] V. Paxson, "Strategies for sound Internet measurement," in *Proc. ACM Internet Measurement Conf. (IMC)*. ACM, 2004, pp. 263–271, physical impossibility as grounds for discarding a trace.

[2] Standard Performance Evaluation Corporation, "SPECjbb2015 run and reporting rules, v1.02," 2018, §3.4.5; an initial clock offset beyond ten minutes voids the run.

[3] J. von Kistowski, J. A. Arnold, K. Huppler, K.-D. Lange, J. L. Henning, and P. Cao, "How to build a benchmark," in *Proc. 6th ACM/SPEC Int. Conf. on Performance Engineering (ICPE)*, 2015, pp. 333–336, section 3, the five quality criteria of a standardized benchmark.

[4] A. Sharma, D. Shah, D. Lariviere, and H. ElBakoury, "Time, causality, and observability failures in distributed AI inference systems," *arXiv preprint arXiv:2604.21361*, Apr. 2026, open Compute Project, Unified Intelligent Infrastructure workstream.

[5] M. Kuperberg, M. Krogmann, and R. H. Reussner, "Metric-based selection of timer methods for accurate measurements," in *Proc. 2nd ACM/SPEC Int. Conf. Performance Engineering (ICPE)*, Mar. 2011, pp. 151–156.

[6] OpenMessaging Project, Linux Foundation, "The OpenMessaging benchmark framework," 2018, available at openmessaging.cloud/docs/benchmarks/; accessed 19 August 2026.

[7] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2009, pp. 265–276.

[8] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*. ACM, 2007, pp. 57–76.

[9] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 2015, pp. 73:1–73:12.

[10] Index.dev, "NATS vs Redis vs Kafka: message broker comparison," Practitioner comparison, 2026, https://www.index.dev/skill-vs-skill/nats-vs-redis-vs-kafka; gives Kafka "sub-10ms latency at p99" and gives Redis and NATS no percentile at all, only "sub-millisecond". No retained fraction, rejection count, timestamp resolution or send rate. Read 2026-09-03. (The correction to an earlier reading of this page is recorded in the supplement's S1.).

[11] Apache Software Foundation, "Apache Kafka tools — `EndToEndLatency.java` and `ProducerPerformance.java`," Source repository, 2026, https://github.com/apache/kafka; percentiles are computed from integer-divided milliseconds. Read 2026-08-31.

[12] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking distributed stream data processing systems," in *Proc. IEEE 34th Int. Conf. Data Engineering (ICDE)*, 2018, pp. 1507–1518.

[13] G. Van Dongen and D. Van den Poel, "Evaluation of stream processing frameworks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 8, pp. 1845–1858, 2020.
[14] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 1978.
[15] IEEE, "IEEE standard for a precision clock synchronization protocol for networked measurement and control systems," IEEE Std 1588-2019, 2020, revision of IEEE Std 1588-2008.
[16] S. Yang, J. Jeong, B. Scholz, and B. Burgstaller, "Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads," *arXiv preprint arXiv:2205.09325*, 2022.
[17] M. Kogias, S. Mallon, and E. Bugnion, "Lancet: A self-correcting latency measuring tool," in *Proc. USENIX Annual Technical Conf. (ATC)*, 2019, pp. 881–896, §4.3: terminates and reports the results with an inconclusive verdict.
[18] R. Kundel, "P4STA: High precision network function benchmarking," in *Accelerating Network Functions Using Reconfigurable Hardware*, ser. Springer Theses. Springer Nature Switzerland, 2024, pp. 93–115.
[19] B. Villain, M. Davis, J. Ridoux, D. Veitch, and N. Normand, "Probing the latencies of software timestamping," in *Proc. IEEE Int. Symp. Precision Clock Synchronization for Measurement, Control and Communication (ISPCS)*, 2012, pp. 1–6, dTrace breakdown against a DAG hardware reference.
[20] V. Paxson, "On calibrating measurements of packet transit times," in *Proc. ACM SIGMETRICS*. ACM, 1998, pp. 11–21.
[21] Jaeger contributors, "Clock skew adjuster considered harmful (issue #1459)," Issue tracker, 2019, https://github.com/jaegertracing/jaeger/issues/1459; the adjustment has shipped disabled since v1.20.0 (2020), which set `-query.max-clock-skew-adjustment` to `0s`. Read 2026-08-21.
[22] A. Swami and N. Chougule, "Architecture dependent temporal observability under deployment interference in edge inference systems," *arXiv preprint arXiv:2605.17701*, 2026.
[23] OpenMessaging Benchmark contributors, "Can the average publish latency be larger than end-to-end latency? (issue #216)," https://github.com/openmessaging/benchmark/issues/216, 2021, the anomaly attributed to inter-node skew, on a same-node report.
[24] J. C. Gust, R. M. Graham, and M. A. Lombardi, "Stopwatch and timer calibrations (2009 edition)," National Institute of Standards and Technology, NIST Special Publication 960-12, 2009, uncertainty budget carrying both operator reaction time and display resolution.
[25] V. Paxson, G. Almes, J. Mahdavi, and M. Mathis, "Framework for IP performance metrics," Internet Engineering Task Force, RFC 2330, May 1998.
[26] V. Raisanen, G. Grotefeld, and A. Morton, "Network performance measurement with periodic streams," Internet Engineering Task Force, RFC 3432, Nov. 2002.
[27] G. Tene, "How NOT to measure latency," 2015, talk, Strange Loop / QCon; coined "coordinated omission".
[28] C. Millsap and J. Holt, *Optimizing Oracle Performance*. Sebastopol, CA: O'Reilly, 2003, section 7.7, "Quantization Error".
[29] Hewlett-Packard, "Time interval averaging," Hewlett-Packard Company, Application Note 162-1, 1970, part no. 02-5952-0766; undated; the year is its *HP Journal* citation.
[30] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, "The Linux scheduler: a decade of wasted cores," in *Proc. 11th European Conf. Computer Systems (EuroSys)*. ACM, 2016, pp. 1–16.
[31] A. Chandrasekar and J. Kramberger, "Identifying and mitigating systemic measurement bias in production LLM inference benchmarks," *arXiv preprint arXiv:2605.24217v2*, 2026.
[32] StatsBomb, "Statsbomb open data," *GitHub Repository*, 2023. [Online]. Available: https://github.com/statsbomb/open-data
[33] Y. Virkar and A. Clauset, "Power-law distributions in binned empirical data," *The Annals of Applied Statistics*, vol. 8, no. 1, pp. 89–119, 2014.
[34] S. Guo, "Avoid adding negative metric values (pull request #56)," OpenMessaging Benchmark, commit ebb5d6b8, Mar. 2018, https://github.com/openmessaging/benchmark/pull/56; accessed 18 August 2026.
[35] Apache Pulsar contributors, "`PerformanceConsumer.java`," Source repository, 2026, https://github.com/apache/pulsar. Read 2026-08-21.
[36] Broadcom, "RabbitMQ PerfTest — `Consumer.java`," Source repository, 2026, https://github.com/rabbitmq/rabbitmq-perf-test; passes the difference unfiltered. Read 2026-08-31.
[37] Synadia, "NATS cli — `cli/latency_command.go`," Source repository, 2026, https://github.com/nats-io/natscli; appends the difference unconditionally. Read 2026-08-31.
[38] IOP Systems, "Rezolus: systems performance telemetry — `scheduler_discarded_samples`," Source repository, 2026, https://github.com/iopsystems/rezolus; counts samples discarded for out-of-order timestamps. Read 2026-08-20.
[39] Apache Kafka contributors, "KAFKA-19888: Clamp negative values in coordinator histograms," Issue tracker and pull request #20986, 2025, https://issues.apache.org/jira/browse/KAFKA-19888; merged 2025-11-26, shipped in 4.2.0, backported to 4.0.2/4.1.2. Read 2026-08-21.
[40] B. Gregg, "Linux bcc/BPF run queue (scheduler) latency," https://www.brendangregg.com/blog/2016-10-08/linux-bcc-runqlat.html, Oct. 2016, introduces `runqlat`; reports the distribution as bimodal without attributing a mechanism.
[41] D. Bristot de Oliveira, D. Casini, J. Lelli, and T. Cucinotta, "Timerlat: Real-time Linux scheduling latency measurements, tracing, and analysis," *IEEE Trans. Comput.*, vol. 74, no. 8, pp. 2608–2620, 2025.
[42] V. Guittot, "`sched/fair`: Manage lag and run to parity with different slices," Linux kernel mailing list, Jul. 2025, states the base-slice protection and the slice-plus-tick lag bound.
[43] Z. Zhou, "sched: Reduce the default slice to avoid tasks getting an extra tick," Linux kernel mailing list, commit 2ae891b82695, 2025, reviewed-by V. Guittot; reduces the constant to `0.70` in v6.15.
[44] S. Shalunov, B. Teitelbaum, A. Karp, J. W. Boote, and M. J. Zekauskas, "A one-way active measurement protocol (OWAMP)," Internet Engineering Task Force, RFC 4656, Sep. 2006.
[45] Linux Kernel Documentation, "KVM-specific MSRs: `MSR_KVM_SYSTEM_TIME_NEW`," https://docs.kernel.org/virt/kvm/x86/msr.html, 2026, flag bit 0, CPUID `0x40000001` bit 24. Read 2026-08-21.

Gustavo Pedro Ricou received the B.Sc. degree in physics from the Universidade do Porto, Portugal, and specialized in nuclear fusion science at the Universität Stuttgart, Germany, and the Université de Lorraine, France. He holds an M.Sc. in software design with cloud-native computing from the Technological University of the Shannon, with first-class honors, and is currently pursuing an M.Sc. in statistics and data science at Trinity College Dublin, Ireland. He is a Software Engineer with BoscaSports. His research concerns measurement validity — what a reported number is really evidence of — in computer systems, environmental measurement and sports analytics, using pre-registration and physical-consistency auditing.

David Gregg received the Ph.D. degree in computer science from the Technische Universität Wien, Vienna, Austria, in 2001, for work on compiling for instruction-level-parallel architectures. He is a Professor in the School of Computer Science and Statistics, Trinity College Dublin, Ireland, and a Fellow of the Irish Computer Society. His research concerns compilers, algorithms and software optimization for multicore and vector computers, and low-resource techniques for deep neural networks.

Romaric Duvignau received the Ph.D. degree in computer science from the University of Bordeaux, France, in 2015, for work on the maintenance and simulation of dynamic random graphs. He is an Associate Professor and Head of the Network and Systems unit at Chalmers University of Technology, Gothenburg, Sweden. His research concerns distributed and data-driven algorithms, the monitoring of large-scale distributed systems, and stream analytics for cyber-physical systems.

Nikolas Herbst received the Ph.D. degree from the University of Würzburg, Germany, in 2018. He is a research group leader at the Chair of Software Engineering, University of Würzburg, and serves as elected vice-chair of the SPEC Research Cloud Group. His research topics include predictive data analysis, elasticity, auto-scaling, resource management, and performance evaluation of virtualized environments.